

Comprehensive Introspection via Preimage Verification: Architecture, Enforcements, and Cryptographic Extensions of OP_PUSH_TX

The Bitcoin Script execution environment was deliberately designed as a stateless, non-Turing complete stack machine¹. To preserve transaction validation parallelism and prevent unbounded computational loops, protocol designers omitted native introspection opcodes that would allow an executing script to inspect the transaction context in which it operates¹. Consequently, locking scripts (scriptPubKey) were historically restricted to isolated cryptographic validations, such as verifying digital signatures against static public keys or checking hash preimages¹.

The discovery and formalization of the transaction introspection technique known as OP_PUSH_TX fundamentally altered this computational model². Rather than relying on soft-forked introspection opcodes, OP_PUSH_TX requires the spending entity to provide the complete signature hash (sighash) preimage of the spending transaction as an unlocking parameter on the execution stack². The script then synthesizes an Elliptic Curve Digital Signature Algorithm (ECDSA) signature from this preimage directly within the execution engine and verifies it against a deterministically known public key using OP_CHECKSIG or OP_CHECKSIGVERIFY².

Because the consensus engine rejects any transaction whose signature digest does not match the double-SHA256 hash of the transaction currently being validated, successful execution provides an absolute cryptographic guarantee that the pushed stack item is an exact byte-for-byte representation of the active transaction¹. Once this preimage is validated on the stack, the contract can parse, isolate, and evaluate any structural field of the spending transaction². This capability transforms the UTXO model into a stateful, verifiable computation platform capable of enforcing arbitrary covenants, recursive state machines, and zero-knowledge validity proofs¹.

Foundational Mechanics of the OP_PUSH_TX Primitive

The Sighash Preimage Structure

The transaction introspection mechanism operates by validating the serialization format established by replay-protected signature hash algorithms, such as those derived from BIP-143¹. When an input evaluates an opcode such as OP_CHECKSIG, the consensus validation client constructs a canonical serialization representing the spending context¹.

The canonical sighash preimage consists of ten distinct serialized fields:

Field Index	Field Name	Type / Length	Description and Consensus Semantics
1	nVersion	4-byte int32_t	Transaction format version number; governs consensus feature flags ³ .
2	hashPrevouts	32-byte uint256	Double-SHA256 hash of the serialization of all input outpoints ³ .
3	hashSequence	32-byte uint256	Double-SHA256 hash of the serialization of nSequence values for all inputs ³ .
4	outpoint	36 bytes	Specific outpoint spent by this input, comprising a 32-byte TXID and a 4-byte index ³ .
5	scriptCode	Variable byte string	Serialized locking script being executed, prefixed by its length ³ .
6	value	8-byte int64_t	Explicit satoshi value contained within the UTXO currently being consumed ³ .
7	nSequence	4-byte uint32_t	Sequence number assigned to this specific input; governs relative

			locks ³ .
8	hashOutputs	32-byte uint256	Double-SHA256 hash of the serialization of all output structures and values ³ .
9	nLocktime	4-byte uint32_t	Absolute transaction-level block height or Unix timestamp maturation lock ³ .
10	sighashType	4-byte uint32_t	Signature hash type byte, extended with zero-padding to 4 bytes ³ .

This complete serialized byte string is denoted as $\mathcal{P}_{stx}$ ¹. The consensus digest e against which signature validation operations are computed is defined mathematically as:

$$e = \text{SHA256}(\text{SHA256}(\mathcal{P}_{stx}))$$

In-Script Signature Synthesis

Under conventional spending schemes, digital signatures are calculated off-chain using the spender's private key and supplied as witness data². In OP_PUSH_TX, the locking script synthesizes the signature dynamically during script execution².

An ECDSA signature over the secp256k1 curve with base point G and group order n requires selecting an ephemeral nonce $k \in [1, n - 1]$, deriving the curve point $R = k \cdot G = (r_x, r_y)$, and calculating the signature pair (r, s) according to:

$$r = r_x \pmod{n}$$

$$s = k^{-1}(e + r \cdot d) \pmod{n}$$

where d represents the private key associated with public key $P = d \cdot G$ ⁷. To minimize stack operations and avoid non-native elliptic curve multiplication in Script, the contract adopts a known, trivial keypair where both the private key and the ephemeral nonce are fixed

constants¹. By assigning:

$$d = 1 \implies P = 1 \cdot G = G = (G_x, G_y)$$

$$k = 1 \implies R = 1 \cdot G = G \implies r = G_x \pmod n$$

the computation of s collapses to a single modular addition:

$$s = 1^{-1}(e + G_x \cdot 1) \pmod n = (e + G_x) \pmod n$$

Because G_x is an immutable secp256k1 parameter and e is computed directly on the stack via OP_HASH256 from the supplied preimage $\mathcal{P}_{stx}$, evaluating s requires only scalar transformation and modular arithmetic⁷. The contract packages the resulting scalar values (r, s) into an ASN.1 Distinguished Encoding Rules (DER) byte sequence, appends the intended sighashType byte, pushes the base point G , and triggers OP_CHECKSIGVERIFY⁷. If the stack item $\mathcal{P}_{stx}$ diverges from the spending transaction, the digest calculated by the host node will fail to match e , producing an invalid signature and immediately halting script execution¹.

Script Optimizations and Low-S Compliance

The implementation of in-script signature synthesis must address script engine quirks and cryptographic consensus rules⁷. Historically, script arithmetic operated on variable-length signed-magnitude integers, complicating 256-bit operations⁷. This resulted in two primary variations of the introspection technique:

Implementation Characteristic	Optimal OP_PUSH_TX Script	Classic OP_PUSH_TX Script
Bytecode Footprint	Compact; consumes under 100 bytes of script code ⁷ .	Verbose; consumes several hundred bytes of script code ⁷ .
Stack Mechanics	Uses OP_BIN2NUM, OP_1ADD, and fixed 0x20 OP_NUM2BIN padding ⁷ .	Employs repetitive OP_SPLIT, OP_SWAP, and OP_CAT byte manipulation loops ⁷ .

Low-S Enforcement	Unchecked in-script; delegates Low-S compliance to off-chain tx malleation ⁷ .	Evaluates $s > (n -$ in-script; negates to $s' = n -$ via arithmetic opcodes ⁷ .
Execution Failure Rate	Approximately 50% failure rate per un-malleated transaction candidate ⁷ .	Drops failure rate to approximately $1/256$, limited only by DER length edge cases ⁷ .
Typical Use Case	Highly optimized smart contracts where transaction fees must be minimized ⁷ .	Environments where off-chain transaction mining/malleation is computationally costly ⁷ .

The BIP-62 consensus standard mandates that the s -scalar of a DER signature must occupy the lower half of the curve order:

$$s \leq \frac{n-1}{2}$$

This rule prevents transaction malleability⁷. Because the message digest e is uniformly distributed, computing $s = (e + G_x) \pmod n$ yields an upper-half scalar 50% of the time⁷. The Optimal OP_PUSH_TX script accommodates this constraint by requiring off-chain software to iteratively malleate transaction fields—such as varying the nLocktime or adjusting dummy change amounts—until a hash yielding an in-script Low-S scalar is achieved⁷. Conversely, the Classic OP_PUSH_TX script explicitly performs endianness inversion and group-order subtraction directly on the execution stack, guaranteeing compliance at the expense of substantial script size⁷.

Preimage Byte Reduction via OP_CODESEPARATOR

A significant challenge when evaluating OP_PUSH_TX within complex smart contracts is the size of the fifth field, scriptCode, which by default replicates the entire executing scriptPubKey⁸. For contracts containing intricate state validation logic, pushing this full preimage increases unlocking transaction sizes and associated network relay fees⁸.

The consensus semantics of OP_CODESEPARATOR provide a mechanism to truncate this data⁸. When forming the sighash preimage, the consensus rules specify that all bytecode executed prior to and including the most recent OP_CODESEPARATOR must be removed from scriptCode¹. By placing an OP_CODESEPARATOR directly before the signature checking routine, the contract causes scriptCode to encompass only the final signature verification

stub¹. As demonstrated in high-level contract languages such as sCrypt via the checkPreimageOCS interface, this optimization cuts preimage payload requirements from over 900 bytes down to approximately 201 bytes, achieving an approximate 80% reduction in bytecode overhead⁸.

State Continuity and Recursive Covenant State Machines

Recursive Covenants and State Accumulation

The primary utility of verified transaction introspection is the enforcement of recursive covenants, which are spending rules requiring that funds be transferred exclusively to an identical or functionally evolved contract instance in the subsequent transaction generation³. Because transactions irrevocably consume their inputs, state persistence across time requires rebuilding and projecting the contract's updated state into newly created outputs³. This architecture allows the underlying ledger to operate as an unbounded ticker tape, supporting stateful computation where contract states transition deterministically from block to block³.

A stateful covenant transaction operates across successive stages. In generation N , the input script validates the preimage $\mathcal{P}_{stx}$, extracts the current state S_N from the executing script code, and computes the state transition $S_{N+1} = f(S_N, \text{args})$ ³. The script then constructs a new locking script containing the updated state S_{N+1} , encapsulates it within an output specification, and verifies that the double-SHA256 digest of this output matches the hashOutputs field in $\mathcal{P}_{stx}$ ¹². When this transaction confirms, output 0 becomes the contract's new state instance S_{N+1} , which will repeat the identical introspection and transition pipeline in generation $N + 1$ ³.

Dynamic State Morphing

To implement this recursive lifecycle, a contract's bytecode is divided into two distinct components: an invariant logic segment ($\mathcal{C}_{logic}$) and a mutable state payload ($\mathcal{S}_{state}$)³. During execution, the contract pulls scriptCode from the verified preimage and invokes OP_SPLIT to decouple $\mathcal{C}_{logic}$ from the serialized parameters of $\mathcal{S}_t$ ¹².

After business rules evaluate the legitimacy of the transition, the script recalculates $\mathcal{S}_{t+1}$, concatenates it with $\mathcal{C}_{logic}$ using OP_CAT, prepends the required satoshi balance along with output length headers, and compares the resulting output hash against hashOutputs¹². This

dynamic state morphing allows UTXOs to function as decentralized state machines, handling data structures such as distributed Merkle accumulators, token ledgers, and games of arbitrary complexity entirely on-chain¹².

Output-Level Enforcements and Flow Control

By dissecting the eighth field of the verified preimage, hashOutputs, a contract establishes strict constraints over transaction destinations, satoshi distributions, and downstream script structures².

Recipient Whitelisting and Destination Locking

The hashOutputs commitment represents the double-SHA256 hash of all concatenated outputs, each serialized as an 8-byte satoshi amount, a variable-length script length prefix, and the output scriptPubKey³. A smart contract can restrict fund flow to a static whitelist of public key hashes or designated multisig scripts by assembling the required output template in memory, computing its double-SHA256 digest, and asserting equivalence to hashOutputs¹. This construct guarantees that assets cannot be redirected to unauthorized destinations, forming the foundation of non-custodial escrows, programmable inheritance schemes, and regulatory compliance covenants³.

Value Preservation, Splits, and Mining Fee Controls

Because native Script lacks direct introspection of network fees, unconstrained transactions are vulnerable to attacks that drain contract funds into excessive miner fees⁹. OP_PUSH_TX mitigates this vulnerability through precise mathematical accounting:

Under strict value preservation, the contract extracts the input satoshi amount from field 6 (value) and asserts that Output 0 contains the exact same satoshi balance, forcing the transacting party to supply all mining fees via secondary inputs³. For dividend distribution or revenue-sharing arrangements, the contract evaluates integer arithmetic over field 6, enforcing proportional splits:

$$V_{out,j} = \left\lfloor \frac{V_{in} \cdot \beta_j}{10000} \right\rfloor$$

across designated recipient outputs³. Furthermore, the contract can establish an upper ceiling on mining fees by extracting V_{in} , parsing each output value $V_{out,j}$ that forms hashOutputs, and confirming:

$$V_{in} - \sum_j V_{out,j} \leq Fee_{max}$$

This prevents fee-sniping and protects automated contracts from programmatic fee-draining attacks⁹.

Provable Token Burning

Decentralized cross-chain bridges utilizing the burn-and-mint paradigm require provable destruction of assets on the host ledger prior to synthetic minting on a destination chain¹. OP_PUSH_TX enforces provable burning on-chain¹. By inspecting hashOutputs, the contract verifies that Output 0 has an assigned value of zero satoshis and that its locking script begins with OP_FALSE OP_RETURN¹. Consensus validation rules designate any output governed by OP_FALSE OP_RETURN as strictly unspendable, permanently eliminating the token from the consumable UTXO set and providing a verifiable on-chain burn event¹.

Output Structure and Dynamic Change Safeguards

When a smart contract must emit dynamic change to a transacting caller while preserving its primary capital, it validates output arrangements by calculating exact byte lengths⁹. The contract asserts that the transaction contains at most two outputs: Output 0, which preserves the contract's balance and state, and Output 1, which routes remaining funds back to the spender⁹. By checking the length and structure of the components comprising hashOutputs, the contract prevents attackers from attaching unauthorized outputs to divert funds³.

Input-Level Structural Constraints and Network Security

Beyond output validation, OP_PUSH_TX enables comprehensive analysis of the transaction's input vector through fields 2 (hashPrevouts), 3 (hashSequence), 4 (outpoint), and 7 (nSequence)².

Single-Input Mandates and Replay Attack Mitigation

A significant threat to UTXO covenants is the multi-input consolidation or replay attack¹. If a covenant contract restricts only Output 0, a malicious actor could assemble a transaction spending multiple identical covenant UTXOs simultaneously¹. In this scenario, the single Output 0 satisfies the localized covenant checks of all consuming inputs, effectively destroying multiple token UTXOs or state machines while creating only one successor output¹.

OP_PUSH_TX neutralizes this vulnerability by enforcing a strict single-input constraint¹:

1. The contract reads the executing input's 36-byte outpoint from field 4 of the verified preimage².
2. The contract calculates the double-SHA256 digest of this single outpoint¹.
3. The script compares the resulting hash against hashPrevouts (field 2)¹.

Because hashPrevouts commits to the serialized concatenation of *all* input outpoints, the statement:

$$\text{hashPrevouts} \equiv \text{SHA256}(\text{SHA256}(\text{outpoint}))$$

holds true if and only if the transaction contains exactly one input¹. If an attacker injects a secondary input, the length and contents of the preimage change, causing hashPrevouts to diverge from the single-outpoint hash and triggering an immediate script failure¹.

Sibling Input Introspection and Transaction Latching

Conversely, multi-asset coordination protocols—such as the BOLT and TSL1 token architectures—require multi-input verification where an input asserts that an adjacent input supplies an authorized identity badge, administrative token, or specific payment UTXO⁴. In this pattern, the spender supplies the entire concatenated array of input outpoints as an unlocking script parameter⁴. The contract computes the double-SHA256 hash of this array, asserts equality against hashPrevouts, and uses OP_SPLIT to parse sibling input TXIDs and indices⁴. This establishes trustless transaction latching, allowing decentralized state machines to verify adjacent transactional context atomically¹⁸.

Emulation of Temporal Primitives

In network environments or legacy node software where dedicated time-checking opcodes such as OP_CHECKLOCKTIMEVERIFY (BIP-65) or OP_CHECKSEQUENCEVERIFY (BIP-112) are disabled or unavailable, OP_PUSH_TX provides complete functional emulation²:

Absolute locktimes are enforced by reading nLocktime (field 9) directly from the stack, converting it into a numeric scalar, asserting that it exceeds a required block height or timestamp threshold, and checking that nSequence (field 7) is strictly less than 0xFFFFFFFF to ensure locktime enforcement².

Similarly, relative locktimes are emulated by evaluating nSequence directly, parsing its relative delay flags, and confirming that the input has attained the required maturation period before allowing execution to proceed³.

Sighash Flag Permutations and Collaborative Multi-Party Protocols

The introspection scope of OP_PUSH_TX is governed by the sighashType byte embedded in field 10¹. Modifying this flag changes the hashing rules for hashPrevouts, hashSequence, and hashOutputs, enabling diverse multi-party coordination patterns¹.

SIGHASH_ALL (0x01 / 0x41)

Commits to: All Inputs, All Outputs, nLocktime, nVersion

Scope: Complete transaction immutability; strict self-contained covenants

SIGHASH_ANYONECANPAY | SIGHASH_ALL (0x81)

Commits to: Executing Input Only, All Outputs, nLocktime, nVersion

Scope: Dynamic fee sponsorship; callers can attach secondary funding inputs

SIGHASH_SINGLE (0x03 / 0x43)

Commits to: All Inputs, Identically Indexed Output Only

Scope: Pipeline batching and non-interactive order-book DEX execution

SIGHASH_NONE (0x02 / 0x42)

Commits to: All Inputs, Zero Outputs

Scope: Delegated output routing and competitive auction liquidations

SIGHASH_ANYONECANPAY: Caller-Funded State Execution

Under standard SIGHASH_ALL (0x01 or 0x41), hashPrevouts binds to every input in the transaction, preventing third parties from adding fee inputs without invalidating the contract's preimage¹.

Configuring the contract to evaluate via Tx.checkPreimageAdvanced with:

`sighashType = SIGHASH_ANYONECANPAY | SIGHASH_ALL (0x81)`

causes hashPrevouts to be zeroed out, committing solely to the outpoint of the executing input⁹. This facilitates caller-funded autonomous contracts⁹.

A public state machine, such as an on-chain counter or registry, can maintain its internal balance intact⁹. An external user triggers a state advance by appending their personal UTXO as Input 1 and including a change address as Output 1, covering the transaction's miner fees⁹. The contract's internal logic asserts that Output 0 continues to hold the contract's invariant balance while incrementing its state⁹.

Assurance Contracts and Crowdfunding Pledges

The combination of SIGHASH_ANYONECANPAY and output introspection enables trustless crowdfunding covenants³. Pledging participants sign inputs committing their satoshis to a

transaction where hashOutputs requires Output 0 to deliver the target funding total M to a campaign beneficiary³.

Because each pledge commits to the finalized recipient outputs but allows arbitrary additional inputs, the transaction remains invalid and un-broadcastable until enough contributors append

inputs to meet or exceed M ³. If the target is not reached, the campaign fails automatically, and individual contributors can reclaim their funds without reliance on an intermediary³.

SIGHASH_SINGLE: Pipeline Processing and Decentralized Order Books

When SIGHASH_SINGLE (0x03 or 0x43) is specified, the preimage's hashOutputs field commits

solely to the output sharing the same numeric index as the executing input¹.

This allows batched settlement pipelines where distinct, independent smart contracts execute in parallel across different input/output slots in the same transaction without interfering with each other's output hashing structures¹.

Furthermore, this flag enables non-interactive limit-order book swaps¹. A trader locks Token A at input index 0 and signs using SIGHASH_SINGLE, mandating that Output 0 must send a specified quantity of Token B to their personal address¹. An arbitrageur or automated market maker can complete the swap by supplying Token B at input index 1, receiving Token A , and broadcasting the completed atomic trade to the network¹.

SIGHASH_NONE: Delegated Processing and Arbitrary Liquidation

Evaluating OP_PUSH_TX under SIGHASH_NONE (0x02 or 0x42) excludes all outputs from the preimage commitment¹. In this mode, the contract enforces spending preconditions exclusively on the input vector, state variables, or external proofs, leaving output construction unconstrained¹.

This mechanism is applied in decentralized lending and collateral liquidation engines¹. When an account becomes under-collateralized, a liquidation contract allows any third-party bot to seize the position, provided the input conditions demonstrate solvency failure¹. The liquidator dynamically defines the output destinations to route recovered principal, capture protocol bounties, and redirect residual capital¹.

Zero-Knowledge Introspection: The Reftx Paradigm and Unbounded Verification

While standard OP_PUSH_TX enables expressive covenants, its capabilities remain bounded by Script's lack of native loops, non-deterministic polynomial evaluations, and data privacy mechanisms¹. The *reftx* framework resolves these constraints by coupling OP_PUSH_TX with on-chain zero-knowledge SNARK verifiers, such as Groth16 pairings implemented in Bitcoin Script¹.

The Reftx Architecture

Rather than running complex computations directly inside Script, the spender delegates validation to an off-chain arithmetic circuit C ¹. To ensure that the zero-knowledge proof verified on-chain cryptographically corresponds to the active spending transaction s_{stx} , the protocol introduces a Transaction Integrity ($\mathcal{TI}$) scheme¹.

During off-chain proof generation, the prover constructs the spending transaction s_{stx} and

calculates an integrity tag τ via the commitment algorithm:

$$\tau = \mathcal{TL}.\text{commit}(s_{stx}, \sigma) = \text{SHA256}(\text{SHA256}(\text{PreSighash}(s_{stx})))$$

The prover then executes a SNARK prover algorithm for an augmented circuit $\mathcal{C}_{\text{ref tx}}$ ¹. This augmented circuit takes public instance parameters $x = (l_{stxo}, u_{stx}, \tau)$ and private witness arguments $w_{\text{aug}} = (s_{stx}, \sigma, w)$ ¹. Inside the arithmetic circuit, the constraints enforce that:

$$\mathcal{TL}.\text{verify}(s_{stx}, \sigma, \tau) = \text{accept}$$

and that the business logic circuit evaluates to true:

$$\mathcal{C}((l_{stxo}, u_{stx}, s_{stx}), w) = 1$$

The generated proof π is submitted to the on-chain locking script:

```
Code snippet
// [Enforce Constraint C via Ref tx]
OP_DUP OP_TOALTSTACK
<< l_stxo >>
<< vk >> [SNARK.verify]
OP_FROMALTSTACK
[Tl.verify]
```

On-chain, the script verifies the proof π against the instance $(l_{stxo}, u_{stx}, \tau)$ using the hard-coded verification key vk ¹. It then executes [Tl.verify], signing the tag τ on the stack using dummy parameters $d = 1$ and $k = 1$, and verifying the signature with OP_CHECKSIG against base point G ¹.

By the knowledge soundness of the SNARK, a valid proof guarantees that the prover knows a valid transaction s_{stx}^* satisfying the circuit constraints¹. By the collision resistance of double-SHA256 in [Tl.verify], that instance must correspond exactly to the live spending transaction:

$$s_{stx}^* \equiv s_{stx}$$

This allows off-chain computation to govern on-chain execution with constant proof verification size¹.

Resolving the Back-to-Genesis Problem

A primary scalability bottleneck in UTXO token schemes is the *back-to-genesis problem*¹.

Verifying the validity of a token at transaction tx_n historically required tracing and evaluating every intermediate transaction back to the initial minting event $tx_{genesis}$ ¹. As the asset changes hands, this transaction history expands indefinitely, eventually exceeding the capacity of on-chain scripts or light client validators¹.

The *reftx* framework resolves this lineage issue by deploying recursive SNARKs or

Proof-Carrying Data (PCD) systems¹. At each transfer $i \rightarrow i + 1$, the spender generates a

recursive proof demonstrating that the prior proof authenticated history back to $tx_{genesis}$, that input 0 spends the verified token output, and that output rules are respected¹. The on-chain contract simply inspects the spending input outputpoint (field 4) using OP_PUSH_TX and runs [SNARK.verify] over the constant-sized proof¹. This compresses an unbounded

transaction graph into an $O(1)$ on-chain verification step, enabling provably compliant, scalable tokens¹.

Advanced Financial, Security, and Cryptographic Primitives

By combining transaction introspection with advanced cryptographic constructions, developers can implement sophisticated financial safeguards and privacy-preserving protocols³.

Covenant-Based Vaults with Timelocked Clawbacks

Institutional asset storage relies on covenant-based vaults to protect funds against private key theft³. The primary Vault UTXO enforces an OP_PUSH_TX constraint mandating that its balance can only be spent into an intermediate Staging UTXO script³.

Once transferred, the Staging UTXO establishes two mutually exclusive spending pathways: The unvaulting pathway enforces a mandatory delay (e.g., 144 blocks, or approximately 24 hours) verified via nSequence or nLocktime introspection, after which funds can be transferred to a standard operational hot wallet³.

The emergency clawback pathway requires no time delay and can be triggered at any point during the unvaulting window by a cold recovery key³. This pathway uses OP_PUSH_TX to enforce that funds must be swept into a deep-cold-storage destination³. If an unauthorized party initiates an unvaulting transaction, the owner detects the event in the mempool or on-chain and triggers the clawback pathway before the timelock expires³.

Decentralized Oracle Integration and In-Script Signature Verification

Smart contracts can ingest external real-world data—such as foreign exchange rates, commodity prices, or real-time event outcomes—without centralized oracles⁶. The oracle publishes data payloads signed using Rabin signatures or ECDSA:

$$\mathcal{M} = (\text{Timestamp} \parallel \text{PriceData})$$

The contract's unlocking script provides $\mathcal{M}$, the signature, and the transaction preimage $\mathcal{P}_{stx}$ ⁶. The script verifies the signature against the oracle's public key directly on the stack⁶. Once the data payload is validated, the contract evaluates the condition (e.g., whether PriceData falls below a collateralization threshold) and uses OP_PUSH_TX to enforce the appropriate outcome³. If the position is solvent, hashOutputs ensures funds remain locked in the covenant; if insolvent, outputs are redirected to execute immediate liquidation³.

Ephemeral Nonce Enforcement and R-Puzzles

Standard signature verification checks the mathematical relation between a static public key and an arbitrary signature⁷. An *R-Puzzle* alters this paradigm by requiring the spending signature to be generated using a specific, pre-committed ephemeral curve point

$$R = k \cdot G \quad \text{or an } r \text{-scalar matching a known hash digest}^9:$$

$$r = \text{Hash}(K_{\text{secret}})$$

Using advanced introspection functions such as Tx.checkPreimageAdvanced, the contract exposes the internal variables of the ECDSA validation pipeline⁹. The contract requires the

spender to reveal the secret ephemeral scalar k , computes the modular inverse k^{-1} , and asserts that the resulting signature satisfies both the transaction introspection check and the R-Puzzle constraint⁹. This construct enables instant, trustless payment channels and cross-chain atomic swaps that do not rely on traditional hash-timelock contracts (HTLCs), avoiding HTLC secret-extraction vulnerabilities⁹.

Zero-Knowledge Contingent Payments (ZKCP)

A Zero-Knowledge Contingent Payment allows the atomic purchase of private data—such as a solution to a computational problem or an decryption key—on-chain without trust²⁰.

The seller generates a cryptographic key K , encrypts the requested data payload, and generates a zero-knowledge proof showing that the encrypted data contains the correct solution and that $H_K = \text{SHA256}(K)$ ²⁰. The buyer deposits funds into a contract

requiring the preimage to H_K alongside an OP_PUSH_TX covenant²⁰.

When the seller claims the bounty, they reveal K in the unlocking script to satisfy the hash lock²⁰. The OP_PUSH_TX covenant verifies that Output 0 routes the payment satoshis to the seller's address while confirming that K is exposed on the public blockchain²⁰. The buyer can then read K directly from the confirmed transaction to decrypt the purchased payload²⁰.

Systematic Taxonomy of OP_PUSH_TX Enforcements

The diverse applications of the OP_PUSH_TX introspection primitive can be categorized by the specific fields of the sighash preimage they evaluate:

Category	Enforcement Capability	Inspected Preimage Fields	Underlying Script Mechanics	Typical Byte Overhead	Primary Security Assumptions
State Continuity	Recursive Covenants & State Accumulators ³	scriptCode, hashOutputs, value	Cleaves state variables from scriptCode, validates transition rules, reconstructs the next scriptPubKey, and asserts equality against hashOutputs ¹² .	Moderate (~150–350 bytes)	Collision resistance of SHA256; Low-S signature compliance ² .
State Continuity	Unbounded Computational State Machines ¹	scriptCode, hashOutputs	Uses the UTXO sequence as an unbounded	Moderate (~200–300 bytes)	Miner transaction relay rules; absence of permanent

			ticker tape, advancing computational state across successive block transactions ³ .		chain reorganizations ¹ .
Output Constraints	Recipient Whitelisting & Escrow Locks ¹	hashOutputs	Precomputes static serialized bytes of allowed destination outputs, hashes them, and checks against hashOutputs ¹ .	Minimal (~80–120 bytes)	Collision resistance of SHA256 ² .
Output Constraints	Strict Fee Ceilings & Anti-Drainin g ³	value, hashOutputs	Subtracts parsed output amounts from input value, verifying that the difference does not exceed a maximum fee limit ⁹ .	Moderate (~120–180 bytes)	Accurate integer bounds handling in Script engine ⁷ .
Output Constraints	Provable Token Burning ¹	hashOutputs	Verifies that Output 0 has a value of zero and	Minimal (~90–130 bytes)	Node consensus rules enforcing

			contains OP_FALSE OP_RETURN metadata, rendering the output provably unspendabl e ¹ .		unspendabil ity of OP_RETURN ¹ .
Output Constraint s	Solvency Proofs & Split Payouts ³	value, hashOutput s	Evaluates proportiona l satoshi splits across multiple outputs and verifies composite hashOutput s ³ .	Moderate (~150–220 bytes)	Arithmetic precision; prevention of integer overflow ⁷ .
Input Constraint s	Single-Input Spending Mandates ¹	hashPrevou ts, outpoint	Hashes the single input outpoint and asserts equivalence to hashPrevou ts, preventing multi-input replay attacks ¹ .	Minimal (~70–100 bytes)	Collision resistance of SHA256 ¹ .
Input Constraint s	Sibling Input Introspectio n ⁴	hashPrevou ts	Receives all transaction outpoints as unlocking parameters, confirms	Moderate to High (~180–300 bytes)	Deterministi c outpoint array serialization ⁴ .

			hash equality to hashPrevouts, and parses sibling inputs ⁴ .		
Temporal Logic	Emulated Absolute Timelocks ²	nLocktime, nSequence	Parses nLocktime directly from the preimage, enforces minimum block height, and verifies that nSequence < 0xFFFFFFFF ² .	Minimal (~80–110 bytes)	Consensus validity of transaction locktimes ² .
Temporal Logic	Relative Timelock Verification ³	nSequence	Extracts nSequence, validates relative height/time flag bits, and checks that minimum delta conditions are satisfied ³ .	Minimal (~80–110 bytes)	Adherence to consensus sequence-lock semantics ³ .
Cooperative Flows	Caller-Funded Gas Contracts ⁹	sighashType, hashOutputs, value	Uses `SIGHASH_ANYONECANPAY`	SIGHASH_ALL` to allow callers to attach secondary	Moderate (~140–200 bytes)

				funding inputs while preserving contract value ⁹ .	
Cooperative Flows	Decentralized Limit Orders & DEX Swaps ¹	sighashType, hashOutputs	Uses SIGHASH_SINGLE to bind input i to output i , permitting external participants to attach matching trade pairs ¹ .	Minimal (~90–130 bytes)	Settlement atomicity across transaction inputs ¹ .
Zero-Knowledge	Zero-Knowledge State Introspection (Reftx) ¹	sighashType, outpoint, scriptCode	Uses an on-chain Groth16 verifier over an integrity tag τ tied to the preimage digest, executing arbitrary NP relations off-chain ¹ .	Very High (~1,500–5,000 bytes)	Knowledge soundness of SNARK; pairing curve security ¹ .
Zero-Knowledge	Compressed Back-to-Genesis Lineage Proofs ¹	outpoint, hashOutputs	Combines recursive zk-SNARKs with OP_PUSH_TX to prove an unbounded	Very High (~2,000–6,000 bytes)	Soundness of recursive SNARK accumulation scheme ¹ .

			transaction ancestry graph in $O(1)$ on-chain time ¹ .		
Advanced Cryptography	Covenant Vaults & Timelocked Clawbacks ³	scriptCode, hashOutputs, nSequence	Enforces a two-phase unvaulting delay, enabling cold storage keys to intercept unauthorized spends before maturation ³ .	Moderate (~200–350 bytes)	Private key security of emergency backup ³ .
Advanced Cryptography	R-Puzzle & Nonce Enforcement ⁹	sighashType, value	Validates that the signature was generated using a specific ephemeral nonce k or curve point R matching an off-chain puzzle ⁹ .	Moderate to High (~180–320 bytes)	Discrete logarithm hardness; secure randomness generation ⁹ .
Advanced Cryptography	Zero-Knowledge Contingent	hashOutputs, value	Forces atomic on-chain	High (~300–800 bytes)	Symmetric cipher integrity;

	Payments (ZKCP) ²⁰		revelation of a cryptographic decryption key conditional on output transfer to the seller ²⁰ .		zero-knowledge soundness ²⁰ .
--	----------------------------------	--	---	--	---

Architectural Conclusions

The formalization of the OP_PUSH_TX technique represents an important architectural shift in UTXO smart contract engineering². By turning the ECDSA signature verification opcode into an in-script introspection engine, it bypasses the historical limitations of Bitcoin Script without requiring network-wide hard forks or dedicated introspection opcodes¹.

By systematically inspecting the ten canonical fields of the sighash preimage, locking scripts can enforce comprehensive transaction-level rules:

- At the output layer, verifying hashOutputs enables recursive covenants, deterministic value splits, strictly capped mining fees, and provable token burning¹.
- At the input layer, inspecting hashPrevouts, outpoint, and sequence fields allows single-input replay attack prevention, sibling input coordination, and emulated timelocks¹.
- Varying the sighashType flag facilitates collaborative multi-party transactions, including open crowdfunding pools, caller-subsidized execution, and decentralized order-book trading¹.

The integration of OP_PUSH_TX with zero-knowledge proof systems via the *reftx* paradigm further expands its capabilities¹. By binding an on-chain integrity tag to an off-chain arithmetic circuit verified via succinct zero-knowledge arguments, contracts can enforce complex, unbounded, and privacy-preserving computations on-chain¹. This resolves fundamental UTXO scalability bottlenecks—such as the back-to-genesis problem—by replacing unbounded transaction history graphs with constant-sized recursive proofs¹. As a result, the combination of transaction preimage introspection and zero-knowledge verification establishes the UTXO model as an expressive, scalable platform for stateful decentralized computation¹.

Works cited

1. op-push.pdf
2. OP_PUSH_TX. One common misconception regarding... | by sCrypt, <https://scryptplatform.medium.com/op-push-tx-3d3d279174c1>
3. OP PUSH TX - Bitcoin Wiki, https://wiki.bitcoinsv.io/index.php/OP_PUSH_TX
4. twostack/tsl1: TSL1: A Peer-to-Peer Token Protocol for Bitcoin - GitHub,

- <https://github.com/twostack/tsl1>
5. OP_PUSH_TX — The Technique and Signing the Pre-Image, https://hub.bsvblockchain.org/higher-learning/bsv-academy/bitcoin-script-new/introduction-to-op_push_tx/op_push_tx-the-technique-and-signing-the-pre-image
 6. Script - Bitcoin Wiki, <https://wiki.bitcoinsv.io/index.php/Script>
 7. 04 - Signing and Checking the Pre-Image | Higher Learning - BSV Hub, https://hub.bsvblockchain.org/higher-learning/bsv-academy/introduction-to-bitcoin-script/chapter-5-op_push_tx/04-signing-and-checking-the-pre-image
 8. op_codeseparator - Medium, <https://xiaohuiliu.medium.com/op-codeseparator-97dc039a8ec8>
 9. Advanced OP_PUSH_TX - sCrypt - Medium, <https://scryptplatform.medium.com/advanced-op-push-tx-78ce84f69a66>
 10. An sCrypt Developer's Guide to Optimization - Medium, <https://xiaohuiliu.medium.com/an-scrypt-developers-guide-to-optimization-bc9848f91e5f>
 11. SpendableStore: A UTXO-based Decentralized Data Store - arXiv, <https://arxiv.org/pdf/2607.02677>
 12. Add current merkle root to records and allow signature ... - GitHub Gist, <https://gist.github.com/zhangweis/f651b7e12acaa7171ab087888e116dc6>
 13. 打开sCrypt的盒子(4)OP_PUSH_TX 原创 - CSDN博客, https://blog.csdn.net/weixin_47461167/article/details/108421502
 14. Multiple Neighborhood Cellular Automata as a Mechanism for, <https://www.mdpi.com/1911-8074/15/8/360>
 15. codenlighten/smartledger-bsv - GitHub, <https://github.com/codenlighten/smartledger-bsv>
 16. GitHub - jd7190/bsv-lock: Tools to manage locking and unlocking, <https://github.com/jd7190/bsv-lock>
 17. dartsv | Dart package - Pub.dev, <https://pub.dev/packages/dartsv>
 18. (PDF) BOLT: A Bitcoin Transaction Latching Mechanism & Token, https://www.researchgate.net/publication/388000992_BOLT_A_Bitcoin_Transaction_Latching_Mechanism_Token_Protocol
 19. Bitcoin Script: OP_CODESEPARATOR - CoinGeek, <https://coingeek.com/op-codeseparator/>
 20. First successful Zero-Knowledge Pay-to-Sudoku Bounty on Bitcoin, <https://coingeek.com/the-first-successful-zero-knowledge-pay-to-sudoku-bounty-on-bitcoin/>